

รายงานการปรับปรุงระบบ LPR Snap

IMPS Service

โปรเจกต์: imps_service

วันที่จัดทำ: 19 พฤษภาคม 2026

ปัญหาที่แก้: Snap ข้ำ และ Snap ผิดค้น (ผิดรถ / รูปซ้ำ)

เวอร์ชันแอป: 1.1.1

สรุปสั้น: ปรับ logic การจับภาพและจับคู่รูปกับข้อมูลรถให้เร็วขึ้นและแม่นยำขึ้น โดยไม่จำเป็นต้องติดตั้ง package ใหม่ สามารถ deploy และ restart service ได้ทันที ค่า `minimum_search`, `maximum_search`, `delay_capture_overview` ในฐานข้อมูลยังใช้ค่าเดิมได้ (มีค่า default อยู่แล้ว)

1. ภาพรวมการทำงานของระบบ Snap

ระบบ IMPS Service ทำงานแบบ 2 ช่องทางหลัก:

- Trigger WebSocket** — เมื่อรถเข้าเลน กล้องส่ง trigger มา → บริการดึงรูปจาก URL กล้อง (LPR + Overview) แล้วบันทึกลงดิสก์และตาราง `snapshots`
- Data WebSocket** — เมื่อ WIM ส่งข้อมูลน้ำหนัก/ความเร็วของรถ → คำนวณรูปที่ตรงเลนและเวลา → OCR → อัปเดต → บันทึกลงตาราง `vehicles`

ปัญหาเกิดเมื่อขั้นตอนที่ 2 มาเร็วกว่าขั้นตอนที่ 1 เสร็จ หรือเมื่อมีรถหลายคันบนเลนเดียวกันในช่วงเวลาสั้น ๆ

2. ปัญหาที่พบ (ก่อนแก้ไข)

อาการ	สาเหตุในโค้ดเดิม
Snap ข้ำ	- เรียก <code>takeSnapshot()</code> แบบไม่ <code>await</code> → ข้อมูลรถมาถึงก่อนรูปบันทึกเสร็จ - หารูปไม่เจอแล้วรอ <code>retry 5 วินาที</code> <code>delay_capture_overview</code> ถูกใช้กับการหา LPR ด้วย (หน่วง 1 วินาทีทุกครั้ง)
Snap ผิดค้น	- SQL ใช้ <code>ORDER BY stamp ASC LIMIT 1</code> → เลือกรูปเก่าสุดในช่วงเวลา ไม่ใช่รูปที่ใกล้เวลารถที่สุด - รูปที่ใช้แล้วไม่ถูก <code>mark</code> → รถคันถัดไปอาจได้รูปเดิม - เลขเลน (<code>lane</code>) เป็น string/number ปนกัน เช่น "1" กับ 1

3. สิ่งที่แก้ไข — รายการไฟล์

ไฟล์	ประเภท	รายละเอียด
src/utils/snapshotRegistry.js	ไฟล์ใหม่	Registry ในหน่วยความจำ จัดรูปตามเวลาที่ใกล้ที่สุด และป้องกันการใช้รูปซ้ำ
src/utils/snapshotManager.js	แก้ไขหลัก	Polling รอรูป, จัดช่วงเวลาใกล้สุด, ลบแถว DB หลังใช้, normalize เลน
src/controllers/DataLogger.js	แก้ไข	await จัดภาพคู่ LPR+Overview, normalize เลนใน trigger, retry 1.5 วินาที
src/controllers/InterComp.js	แก้ไข	เหมือน DataLogger สำหรับ controller แบบ InterComp
src/utils/mappers/mapDataLogger.js	แก้ไขเล็กน้อย	<code>data.lane = Number(rawData.LaneNo)</code>
src/utils/mappers/mapInterComp.js	แก้ไขเล็กน้อย	<code>data.lane = Number(rawData.LaneNo)</code>
.env.example	เอกสาร config	เพิ่ม SNAP_MATCH_POLL_MS , SNAP_MATCH_MAX_WAIT_MS

4. รายละเอียดการแก้ไขแต่ละส่วน

4.1 snapshotRegistry.js (ใหม่)

โมดูลนี้เก็บรายการรูปล่าสุดต่อเลนและประเภท (lpr / overview) ในหน่วยความจำ

- **register()** — บันทึกรูปหลังจับภาพสำเร็จ (lane, type, stamp, path)
- **claimClosest()** — เลือกรูปที่เวลาใกล้ `mappedData.stamp` ที่สุด ภายในช่วง minimum/maximum search และยังไม่ถูกใช้
- **markUsed()** / **isUsed()** — ป้องกันรถคันถัดไปเอารูปเดิม
- **normalizeLane()** — แปลงเลนเป็นตัวเลขเสมอ (แก้ปัญหา string vs number)

4.2 snapshotManager.js

หัวข้อ	ก่อน	หลัง
การจับภาพ	บันทึก DB อย่างเดียว	register ลง memory ทันทีหลังเขียนไฟล์ + บันทึก DB, normalize lane
การหารูป	Query ครั้งเดียว, ORDER BY stamp ASC	Poll ทุก 150ms สูงสุด 3 วินาที; หา memory ก่อน แล้วค่อย DB
การเลือกรูปจาก DB	รูปเก่าสุดในช่วง	ORDER BY ABS(TIMESTAMPDIFF(...)) — รูปใกล้เวลารถที่สุด
delay_capture_overview	ใช้กับทุก type รวม LPR	ใช้เฉพาะ type overview เท่านั้น
หลังใช้รูป	แถวค้างใน DB	DELETE จากตาราง snapshots + mark used ใน memory

4.3 DataLogger.js และ InterComp.js (Trigger)

```
// ก่อน - ไม่รอให้จับภาพเสร็จ
this.lprSnapshotManager.takeSnapshot(url, metadata);
this.overviewSnapshotManager.takeSnapshot(url2, metadata);

// หลัง - รอทั้งคู่พร้อมกัน และ normalize เลน
const lane = normalizeLane(channelId);
await Promise.all([
  this.lprSnapshotManager.takeSnapshot(lprUrl, { ...metadata, type: "lpr", lane }),
  this.overviewSnapshotManager.takeSnapshot(ovUrl, { ...metadata, type: "overview", lane })
]);
```

4.4 Retry เมื่อไม่มีรูปป้าย/overview

	ก่อน	หลัง
เวลารอ retry	5000 ms	1500 ms
หมายเหตุ	การ poll ภายใน findSnapshots ช่วยลดโอกาสต้อง retry อยู่แล้ว	

5. การตั้งค่า (ไม่บังคับต้องเปลี่ยน)

5.1 ค่าในฐานข้อมูล (ตาราง configuration)

ค่า	Default ถ้า NULL	ความหมาย
minimum_search	1000 ms	ค้นหารูปย้อนหลังจากเวลารถได้ไม่เกินเท่านี้
maximum_search	5000 ms	ค้นหารูปล่วงหน้าจากเวลารถได้ไม่เกินเท่านี้
delay_capture_overview	1000 ms	รอก่อนเริ่มหา overview (กล้อง overview มักช้ากว่า LPR)

ใช้งานได้ทันทีโดยไม่แก้ค่าเหล่านี้ — จนเมื่อยังมีอาการเฉพาะ:

- ยังพีดค้นบ่อย → ลด min/max (เช่น 500 / 2000)
- หารูปไม่เจอ → เพิ่ม maximum_search
- overview ว้าง/ช้า → เพิ่ม delay_capture_overview

5.2 ตัวแปรใน .env (ไม่บังคับ)

ตัวแปร	Default	ความหมาย
SNAP_MATCH_POLL_MS	150	ช่วงเวลาระหว่างการลองหารูปซ้ำ
SNAP_MATCH_MAX_WAIT_MS	3000	รอหารูปสูงสุดกี่ ms ต่อครั้ง

6. Flow หลังแก้ไข (แผนภาพ)

```
[Trigger WS] รถเข้าเลน
|
v
await Promise.all( LPR snap + Overview snap )
|
+--> บันทึกไฟล์ + snapshotRegistry.register + INSERT DB
|
[Data WS] ข้อมูลรถ (lane, stamp)
|
v
findSnapshots (poll 150ms, max 3s)
|
+--> claimClosest จาก memory (เวลาใกล้สุด, ยังไม่ used)
|       หรือ query DB แบบ closest timestamp
|
v
mark used + DELETE แถว snapshots
|
v
OCR + Upload (Promise.all เดิม) --> บันทึก vehicles
```

7. ผลที่คาดหวังหลัง Deploy

ด้าน	ผลที่มักเห็น
ความเร็ว	รูปขึ้นเร็วขึ้น; ลดการรอ 5 วินาที; LPR ไม่โดนหน่วง 1 วินาทีจาก overview delay
ความถูกต้อง	จับคู่รูปกับรถตามเวลาใกล้สุด; รูปไม่ถูกใช้ซ้ำระหว่างคัน
Log	ข้อความ "Retrying ... after 5 seconds" น้อยลง หรือเป็น 1.5 วินาที

8. วิธี Deploy

1. ดึงโค้ดล่าสุดไปที่เซิร์ฟเวอร์
2. Restart service: `npm run prod` หรือ `pm2 restart` ตามที่ใช้อยู่
3. (ถ้าต้องการ) เพิ่มใน `.env` : `SNAP_MATCH_POLL_MS`, `SNAP_MATCH_MAX_WAIT_MS`
4. ทดสอบรถจริง 1-2 เลน สังเกตเวลาและความตรงของป้ายทะเบียน

9. ข้อจำกัด / สิ่งที่ยังต้องเช็ด้วยมือ

- **Mapping กล้อง:** ChannelId จาก trigger ต้องตรงกับ LaneNo ใน config `capture_lpr` / `capture_overview` — ถ้า map ผิด โค้ดแก้ logic อย่างเดียวไม่พอ
- **รถชิดกันมากบนเลนเดียว:** อาจต้องลดช่วง `minimum_search` / `maximum_search`

- **ไม่ได้ติดตั้ง package ใหม่** — ใช้ dependency เดิม (axios, sharp, mysql2, dayjs ฯลฯ)

เอกสารนี้สร้างจากการปรับปรุงโค้ด imps_service — LPR Snap Performance Fix

สอบถามเพิ่มเติม: ดูไฟล์ในโฟลเดอร์ src/utils/snapshotRegistry.js และ snapshotManager.js